Semgrep CE로 Android 보안 점검

로컬 룰 2개로 후보를 검출하고, 오탐 분류를 AI로 보강하는 운영 흐름을 정리합니다.

이번 발표의 범위

- 사용 범위: `Semgrep Community Edition CLI` + 직접 작성한 로컬 룰
- 제외 범위: Registry, 외부 rule pack, AppSec Platform/Pro, `p/java`, `p/kotlin`, `p/default`
- 이번 발표에서는 룰 `2개` 만 다룹니다.
- 각 묶음은 같은 순서로 설명합니다.

```
문제 설명
→ 문제 코드
→ 안전한 기본형 / 비교 코드
→ Semgrep 규칙 설명
→ 검출 결과
→ 오탐 사례
→ AI 분류 기준
→ AI 검토 입력
```

로컬 실행 구조

```
android-project/
├── app/src/main/kotlin/com/example/PendingIntentLab.kt
├── app/src/main/java/com/example/LegacyCrypto.java
└── rules/android-local.yml
```

```
semgrep scan --metrics=off \
--config rules/android-local.yml \
app/src/main
```

- 발표용 룰 파일에는 룰 `2개` 만 둡니다.
- 목적은 "완성형 판정기"가 아니라, 후보를 빠르게 추출하고 분류 시간을 줄이는 데 있습니다.

예시 1: 무엇을 점검하나

`android-pendingintent-flag-mutable`

- `PendingIntent`는 단순한 `Intent` 복사본이 아니라, **내 앱 권한으로 나중에 실행할 수 있는 시스템 토큰**에 가깝습니다.
- 다른 앱이나 시스템 UI에 넘겨도, 실행 시에는 원래 `PendingIntent`를 만든 앱 정체성으로 동작할 수 있습니다.
- 그래서 리뷰 포인트는 "이 Intent가 어디로 가는가"뿐 아니라, "누가 나중에 어떤 값을 채워 실행할 수 있는가"까지 보는 데 있습니다.

```
App A creates PendingIntent
-> System UI or App B stores token
-> Later send() / notification action
-> Action runs with App A identity
```

예시 1: 어떤 조합이 위험한가

패턴	왜 위험한가
`FLAG_MUTABLE`을 불필요하게 사용	받은 쪽이 `fillInIntent()`로 미채워진 필드를 바꿀 수 있습니다.
`FLAG_MUTABLE` + empty / implicit `Intent`	action, data, component, extras가 변형될 여지가 커집니다.
`requestCode = 0` 반복 + `FLAG_UPDATE_CURRENT`	기존 토큰과 충돌하거나 extras가 섞일 수 있습니다.
`content://` + URI grant 결합	의도치 않은 URI read/write 권한 흐름으로 이어질 수 있습니다.
민감 작업인데 `FLAG_ONE_SHOT` 없음	삭제, 승인, 결제 같은 1회성 작업이 재실행될 수 있습니다.
notification action이 바로 민감 작업 수행	잠금화면·외부 표면과 결합될 때 오용 여지가 생깁니다.

- 특히 `FLAG_MUTABLE` + implicit `Intent` 조합은 위험도가 높습니다.
- Android 14 / target SDK 34+에서는 이 조합이 기본적으로 막히는 방향으로 강화되었습니다.

예시 1: 코드 리뷰 빨간불 패턴

```
PendingIntent.getActivity(context, 0, Intent(), PendingIntent.FLAG_MUTABLE)
PendingIntent.getBroadcast(context, 0, Intent("SOME_ACTION"), PendingIntent.FLAG_MUTABLE)
PendingIntent.getService(context, 0, intent, PendingIntent.FLAG_MUTABLE)
PendingIntent.getActivity(context, 0, intent, PendingIntent.FLAG_UPDATE_CURRENT)
PendingIntent.getBroadcast(context, 0, intent, PendingIntent.FLAG_CANCEL_CURRENT)
```

- 같이 보면 더 위험한 조건:
  - `setClass`, `setComponent`, `setPackage`가 없음
  - `FLAG_GRANT_READ_URI_PERMISSION` / `WRITE`가 붙음
  - 내부 `Receiver` / `Service`가 extras를 그대로 신뢰함
  - 민감 작업인데 `FLAG_ONE_SHOT`과 만료 개념이 없음

예시 1: 문제 코드

```
package com.example

class PendingIntentLab {
  fun buildDangerousDelete(context: Context, sensitiveFileUri: Uri): PendingIntent {
    val deleteIntent = Intent("com.example.ACTION_DELETE_FILE").apply {
      data = sensitiveFileUri
      addFlags(Intent.FLAG_GRANT_READ_URI_PERMISSION)
      putExtra("fileId", "report-4412")
    }

    return PendingIntent.getBroadcast(
      context,
      0,
      deleteIntent,
      PendingIntent.FLAG_MUTABLE or PendingIntent.FLAG_UPDATE_CURRENT
    )
  }
}
```

- `FLAG_MUTABLE` : 받은 쪽이 미채워진 필드나 extras를 보강할 수 있습니다.
- `Intent("ACTION")` : explicit component가 아니라 implicit `Intent` 입니다.
- `requestCode = 0` : 다른 토큰과 충돌하기 쉬운 기본값입니다.
- `FLAG_UPDATE_CURRENT` : 이전 토큰 보유자도 새 extras로 실행할 수 있습니다.

예시 1: 비교적 안전한 기본형

```
val intent = Intent(context, DeleteConfirmActivity::class.java).apply {
    action = "com.example.ACTION_DELETE_FILE"
    setPackage(context.packageName)
    putExtra("fileId", fileId)
}

val pi = PendingIntent.getActivity(
    context,
    notificationId,
    intent,
    PendingIntent.FLAG_IMMUTABLE or
        PendingIntent.FLAG_UPDATE_CURRENT or
        PendingIntent.FLAG_ONE_SHOT
)
```

- 기본 안전선:
  - explicit component
  - `FLAG_IMMUTABLE`
  - unique `requestCode`
  - 최소 extras
  - 실제 삭제/승인은 대상 화면이나 receiver에서 다시 검증

예시 1: Semgrep 규칙

```yaml
rules:
- id: android-pendingintent-flag-mutable
  languages: [java, kotlin]
  severity: ERROR
  message: >
    Mutable PendingIntent입니다.
    대부분은 FLAG_IMMUTABLE을 사용해야 하며, mutable이 꼭 필요하면
    explicit component/package, unique requestCode, 최소 extras를 함께 검토하십시오.
  patterns:
    - pattern: PendingIntent.get$METHOD($CTX, $REQ, $INTENT, $FLAGS)
    - metavariable-regex:
        metavariable: $FLAGS
        regex: ((?i).*FLAG_MUTABLE.*)
```

- 이 규칙은 의도적으로 넓게 `FLAG_MUTABLE` 사용처를 1차 수집합니다.
- 실제 위험도는 2차 분류에서 봅니다.
- 즉, "mutable이 정말 필요한가", "Intent가 explicit인가", "추가 red flag가 있는가"를 사람이나 AI가 이어서 판단합니다.

예시 1: 검출 결과

```bash
$ semgrep scan --metrics=off \
  --config rules/android-local.yml \
  app/src/main/kotlin/com/example/PendingIntentLab.kt
```

예시 1: 합법적인 mutable 예외 사례

```
object ReplyActionFactory {
  fun buildReply(context: Context, threadId: Int): PendingIntent {
    val replyIntent = Intent(context, ReplyReceiver::class.java).apply {
      action = "com.example.ACTION_INLINE_REPLY"
      setPackage(context.packageName)
    }

    return PendingIntent.getBroadcast(
      context,
      threadId,
      replyIntent,
      PendingIntent.FLAG_MUTABLE or PendingIntent.FLAG_UPDATE_CURRENT
    )
  }
}
```

- notification inline reply처럼 시스템이 `RemoteInput` 결과를 채워야 하는 경우에는 mutable이 실제로 필요할 수 있습니다.
- 이 경우에도 component/package는 고정하고, mutable 필요성이 코드리뷰에서 설명 가능해야 합니다.

예시 1: AI 분류 포인트

1. 이 use case가 inline reply / bubble / alarm 등으로 mutable이 정말 필요한지 확인한다.
2. Intent가 explicit component 또는 package로 고정돼 있는지 확인한다.
3. requestCode가 고유한지, FLAG_UPDATE_CURRENT가 민감 extra를 덮어쓰지 않는지 본다.
4. URI grant, 민감 action, 내부 receiver/service의 extra 신뢰 여부를 함께 본다.
5. 실제로 필요 없는 mutable이면 true positive로, 합법적 시스템 use case면 allowlist 후보로 기록한다.

예시 1: AI 검토 입력 템플릿

```
다음 Semgrep finding이 실제 취약점인지, 오탐인지 Android 코드리뷰 관점에서 판별해줘.

- check_id: android-pendingintent-flag-mutable
- file: app/src/main/kotlin/com/example/PendingIntentLab.kt:12
- rule_intent: mutable PendingIntent 1차 수집
- code:
  return PendingIntent.getBroadcast(
      context,
      0,
      deleteIntent,
      PendingIntent.FLAG_MUTABLE or PendingIntent.FLAG_UPDATE_CURRENT
  )

추가 요청:
1. 실제 취약점 / 오탐 / 추가 확인 필요 중 하나로 분류
2. 그렇게 판단한 근거 3개
3. explicit 여부, requestCode, URI grant, 민감 action 관점에서 더 확인할 call path
4. 규칙 튜닝 포인트 1개와 코드리뷰 코멘트 3줄
```

- sample 1에서는 `FLAG_MUTABLE` 자체보다 implicit `Intent`, `URI grant`, 민감 action이 함께 붙는지까지 봐야 합니다.
- AI에게는 "실제로 mutable이 필요한 use case인가"를 먼저 묻게 하는 편이 분류 효율이 좋습니다.

예시 2: 무엇을 점검하나

`java-android-weak-hash-md5-sha1`

- 점검 대상: `SHA-1`, `MD5`, `HmacSHA1`, `SHA1withRSA` 같은 구식 알고리즘 사용
- 문제 상황: 무결성 검증, 서명 검증, 토큰 서명, 업데이트 검증에 그대로 사용
- 왜 위험한가:
  - 충돌 공격 관점에서 현대 기준에 맞지 않음
  - "레거시라서 그대로" 남아 있는 경우가 많음
  - 보안 결정에 쓰이면 영향이 커짐

```
핵심 질문
- 이 해시는 보안 의사결정에 쓰이는가?
- 단순 표시/호환용인가, 검증용인가?
- SHA-256 이상으로 바꿀 수 있는가?
```

예시 2: 문제 코드

```java
package com.example;

import java.security.MessageDigest;
import java.security.Signature;
import java.util.Arrays;

class LegacyCrypto {
  boolean verifyManifest(byte[] manifest, byte[] expectedDigest) throws Exception {
    byte[] actual = MessageDigest.getInstance("SHA-1").digest(manifest);
    return Arrays.equals(actual, expectedDigest);
  }

  Signature newSigner() throws Exception {
    return Signature.getInstance("SHA1withRSA");
  }
}
```

- `verifyManifest()` 에서는 SHA-1 결과를 실제 비교 판단에 사용합니다.
- `newSigner()` 는 오래된 서명 알고리즘을 그대로 사용합니다.
- 이런 코드는 "레거시 때문에 유지"되는 경우가 많아, 우선 수집이 중요합니다.

예시 2: Semgrep 규칙

```yaml
rules:
- id: java-android-weak-hash-md5-sha1
  languages: [java]
  severity: ERROR
  message: >
    MD5·SHA-1 계열의 약한 해시 또는 서명 알고리즘 사용입니다.
    SHA-256 이상 또는 최신 권장 알고리즘으로 교체하십시오.
  pattern-either:
    - pattern: MessageDigest.getInstance("MD5")
    - pattern: MessageDigest.getInstance("SHA1")
    - pattern: MessageDigest.getInstance("SHA-1")
    - pattern: Mac.getInstance("HmacSHA1")
    - pattern: Signature.getInstance("SHA1withRSA")
    - pattern: Signature.getInstance("SHA1withDSA")
    - pattern: Signature.getInstance("SHA1withECDSA")
```

- `pattern-either` : 해시, HMAC, 전자서명 초기화 지점을 넓게 수집합니다.
- 장점: 단순하고 빠르게 찾을 수 있습니다.
- 한계: "보안 검증"에 쓰는지, "표시용"인지는 구분하지 못합니다.
- 그래서 이 룰은 AI 분류 단계와 함께 쓰기 좋습니다.

예시 2: 검출 결과

```
$ semgrep scan --metrics=off \
  --config rules/android-local.yml \
  app/src/main/java/com/example/LegacyCrypto.java


┌─────────────────────────────┐
│ 2 Code Findings             │
└─────────────────────────────┘

  app/src/main/java/com/example/LegacyCrypto.java
  ❯❯❯ java-android-weak-hash-md5-sha1
      MD5·SHA-1 계열의 약한 해시 또는 서명 알고리즘 사용입니다.
      SHA-256 이상 또는 최신 권장 알고리즘으로 교체하십시오.

      8┆ byte[] actual = MessageDigest.getInstance("SHA-1").digest(manifest);

  ❯❯❯ java-android-weak-hash-md5-sha1
     13┆ return Signature.getInstance("SHA1withRSA");
```

- 분석자는 "이 값이 실제 보안 판단에 쓰이는가"를 먼저 봐야 합니다.
- 이 구분을 사람이 직접 하거나 AI에게 맡기면 분류 속도를 높일 수 있습니다.

예시 2: 오탐 사례

```java
package com.example;

import java.security.MessageDigest;
import java.security.cert.X509Certificate;

class CertificateScreen {
  String legacyFingerprint(X509Certificate cert) throws Exception {
    byte[] fp = MessageDigest.getInstance("SHA-1").digest(cert.getEncoded());
    return Hex.encode(fp);
  }
}
```

- 이 코드는 SHA-1을 쓰지만, 예를 들어 "기존 운영 문서와 맞춰 보기 위한 표시용 지문값"이라면 보안 의사결정일 수도, 아닐 수도 있습니다.
- Semgrep은 사용 목적을 알 수 없기 때문에 우선 후보로 잡는 편이 맞습니다.

예시 2: AI 분류 포인트

```
AI 분류 포인트
1. 해시 결과가 if/allow/deny/verify로 이어지는지 본다.
2. 결과가 UI 표시, 로그, 마이그레이션 보고서로만 가는지 확인한다.
3. 신뢰 판단이 아니라면 낮은 우선순위 또는 호환성 메모로 내린다.
4. 신뢰 판단이라면 실제 취약점으로 분류하고 SHA-256 이상 대체 코드를 제안한다.
```

예시 2: AI 검토 입력 템플릿

```
다음 Semgrep finding이 실제 취약점인지, 오탐인지 Java 코드리뷰 관점에서 판별해줘.

- check_id: java-android-weak-hash-md5-sha1
- file: app/src/main/java/com/example/LegacyCrypto.java:8
- rule_intent: 약한 해시 / 서명 알고리즘 1차 수집
- code:
  byte[] actual = MessageDigest.getInstance("SHA-1").digest(manifest);
  return Arrays.equals(actual, expectedDigest);

추가 요청:
1. 실제 취약점 / 오탐 / 추가 확인 필요 중 하나로 분류
2. 그렇게 판단한 근거 3개
3. 이 값이 trust decision, UI 표시, 호환성 경로 중 어디에 쓰이는지 더 확인할 함수 / call path
4. SHA-256 이상 대체 방향과 규칙 튜닝 아이디어 1개
```

- sample 2에서는 "약한 알고리즘을 썼다"보다 "그 결과가 실제 보안 판단에 쓰이는가"를 먼저 가려내는 것이 중요합니다.
- `check_id`, 호출 코드, 주변 함수, helper 이름까지 함께 주는 것이 중요합니다.

정리

- `PendingIntent`는 나중에 내 앱 정체성으로 실행될 수 있는 토큰이므로, `FLAG_MUTABLE` 사용은 기본적으로 의심하고 봐야 합니다.
- 기본 안전선은 `explicit + immutable + unique requestCode + 최소 extras + receiver 측 재검증`입니다.
- `FLAG_MUTABLE`이 정말 필요한지, implicit `Intent` / URI grant / `FLAG_UPDATE_CURRENT` / 민감 action이 같이 붙는지 확인하는 것이 핵심입니다.
- 약한 해시·서명 탐지는 빠르게 후보를 수집하는 데 유용하지만, 실제 보안 의사결정에 쓰이는지는 별도 확인이 필요합니다.
- Semgrep은 구조 기반 후보 추출에 적합하고, 최종 판정은 코드 문맥 확인과 AI·사람 분류를 함께 써야 정확도가 올라갑니다.